

# Medya Aktarım Uygulaması

Android telefonlardaki fotoğraf ve videoları, veri kaybına karşı güvenli biçimde bilgisayara aktaran bir Windows masaüstü uygulamasının cihaz tarama katmanı

5

gerçek cihazda doğrulandı

291

otomatik test, 205 ms

0

hata, tam tarama

100×

ölçülen soğuk / sıcak farkı

**Hazırlayan:** Can Önen

**Teknoloji:** C# / .NET 10 · Windows Portable Devices (WPD) COM API · MTP protokolü · xUnit

**Kaynak kod:** [github.com/canonencode/Media-Transferring-Application](https://github.com/canonencode/Media-Transferring-Application)

**Tarih:** Eylül 2026

# İçindekiler

|      |                                                         |    |
|------|---------------------------------------------------------|----|
| 1    | Özet                                                    | 3  |
| 2    | Problem Tanımı                                          | 3  |
| 3    | Mimari                                                  | 4  |
| 4    | Çalışma Akışı                                           | 6  |
| 5    | Kullanılan Tasarım Kalıpları                            | 6  |
| 6    | Karşılaşılan Teknik Zorluklar ve Çözümler               | 8  |
| 6.1  | Hata alan klasörün "dosya" sanılması                    | 8  |
| 6.2  | Süre üzerine dört yanlış teori                          | 8  |
| 6.3  | PTP modu: sessiz %23 kayıp                              | 8  |
| 6.4  | Bekçi: kod incelemesinin bulamadığı bug                 | 8  |
| 6.5  | Kurtarılamayan donma                                    | 8  |
| 6.6  | Yanlış sınıflandırmalar                                 | 9  |
| 7    | Test Stratejisi                                         | 10 |
| 8    | Ölçüm Sonuçları                                         | 11 |
| 8.1  | Ana cihazda (Samsung Galaxy A56) başlangıç ve son durum | 11 |
| 8.2  | Cihazlar arası doğrulama                                | 11 |
| 9    | Öğrenilen Dersler                                       | 12 |
| 10   | Sonraki Adımlar                                         | 12 |
| 11   | Geliştirme Süreci ve Çalışma Yöntemi                    | 13 |
| 11.1 | Karar alma                                              | 13 |
| 11.2 | Gerçek donanım ile doğrulama                            | 13 |
| 11.3 | Varsayımların sorgulanması                              | 13 |
| 11.4 | Araçlar ve iş akışı                                     | 13 |
| 11.5 | Öğrenme hedefi                                          | 13 |
| 12   | Yönetici Özeti                                          | 14 |
| 13   | Üç Soruya Doğrudan Cevap                                | 16 |
| 14   | Kod Envanteri                                           | 17 |

|       |                                                |    |
|-------|------------------------------------------------|----|
| 15    | Çalışma Notları — Kavramlar                    | 20 |
| 15.1  | COM — Component Object Model                   | 20 |
| 15.2  | WPD, MTP ve PTP                                | 20 |
| 15.3  | Interop ve çevirmen (RCW, marshalling)         | 21 |
| 15.4  | Referans sayma ve kaynak temizliği             | 21 |
| 15.5  | HRESULT ve COMException                        | 21 |
| 15.6  | Thread'ler, apartment, Interlocked ve Volatile | 22 |
| 15.7  | Devre kesici (Circuit Breaker)                 | 22 |
| 15.8  | Bekçi (Watchdog)                               | 23 |
| 15.9  | Dosya imzası (magic bytes)                     | 23 |
| 15.10 | Birim testi ve saf fonksiyon                   | 23 |
| 15.11 | Soğuk ve sıcak önbellek                        | 24 |
| 15.12 | Arayüz ve bağımlılığın tersine çevrilmesi      | 24 |
| 15.13 | Değişmez veri tipleri (record)                 | 24 |
| 15.14 | Git ve commit mesajı                           | 25 |

## 1 Özet

Bu proje, telefonlardaki fotoğraf ve videoları bilgisayara aktaran ticari araçların (ör. CopyTrans Studio) iki temel zayıflığına cevap olarak başlatıldı: **aktarımın yarıda bozulup hangi dosyanın kopyalandığının bilinmemesi** ve **medyanın onlarca iç içe klasöre dağılmış olması**. Rapor, projenin ilk aşaması olan **cihaz tarama katmanını** kapsar: telefona bağlanan, tüm dosya ağacını yürüten, her dosyayı sınıflandıran ve bunu güvenilir donanım karşısında doğru biçimde raporlayan bileşen.

Çalışma boyunca en belirleyici bulgu şu oldu: **kod incelemesi tek başına yetersizdir**. Beş bağımsız inceleme turunun "sağlıklı" dediği kodda, gerçek cihaz ölçümleri ve otomatik testler 20'den fazla kusur ortaya çıkardı; bunlardan ikisi kullanıcının fotoğraflarını sessizce kaybettiriyordu. Rapor bu kusurları, çözümlerini ve ölçümle doğrulanan sonuçları sunar.

## 2 Problem Tanımı

### Sorun 1 — Kayıp takip

Mevcut araçlar aktarım yarıda kesildiğinde hangi dosyaların tamamlandığını kaydetmez. Kullanıcı ya baştan başlar ya da eksik yedekle kalır.

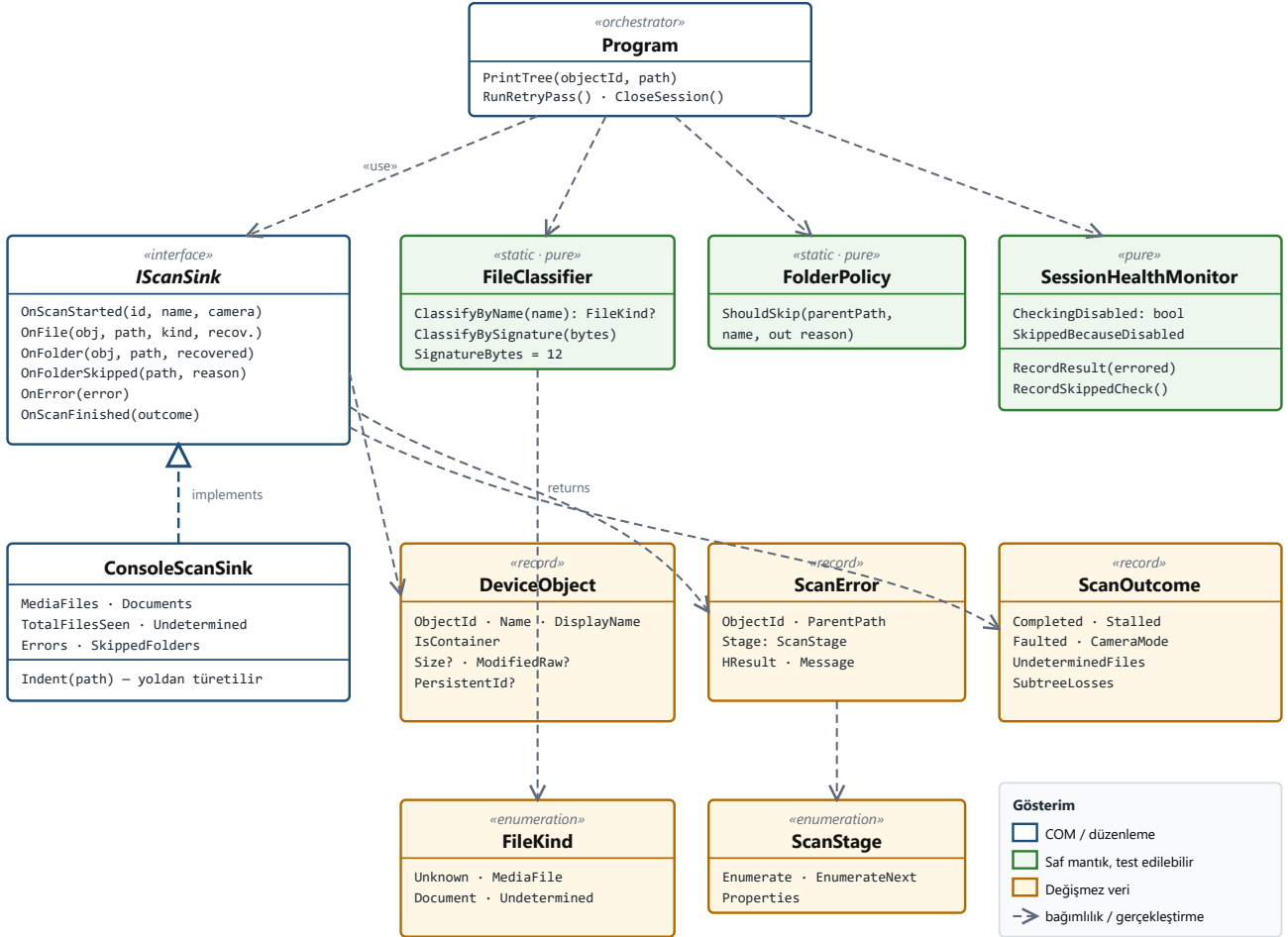
### Sorun 2 — Dağınık yapı

Android medyayı `DCIM/Camera` , `WhatsApp/Media/...` , uygulama klasörleri gibi onlarca yere dağıtır. Elle toplamak pratik değildir.

Her iki sorunun ortak kökü aynıdır: **uygulamanın cihazda ne olduğunu güvenilir biçimde bilmemesi**. Bu yüzden ilk aşama, doğru ve dürüst bir tarama katmanı inşa etmeye ayrıldı. "Dürüst" burada teknik bir gereksinimdir: tarama eksik kaldıysa bunu *söylemelidir*; eksik bir sonucun tam görünmesi, projenin çözmeye çalıştığı hatanın ta kendisidir.

### 3 Mimari

Sistem tek bir sorumluluk ilkesi etrafında ayrıştırıldı: **donanıma dokunan kod ile karar veren kod ayrı yaşar.** Sınıflandırma, klasör politikası ve sağlık izleme saf (pure) sınıflardır — COM, cihaz ya da konsol bilmezler. Bu ayırım olmadan projenin en kritik mantığı, fiziksel telefon bağlı olmadan tek bir satır test edilemiyordu.



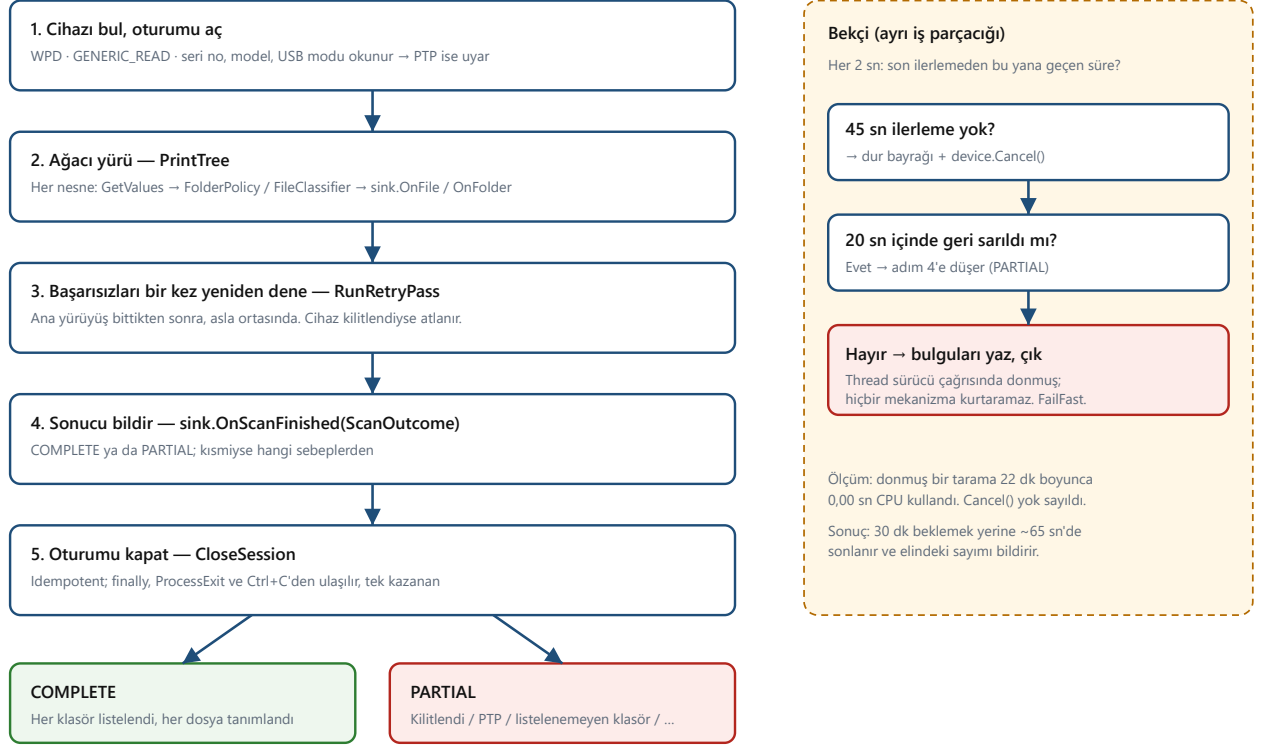
Şekil 1 — UML sınıf diyagramı. Tarama (Program) yalnızca *IScanSink* soyutlamasına bağımlıdır; konsol çıktısı ve gelecekteki SQLite yazıcısı bu arayüzün birer gerçekleştirimidir.

## Bileşenlerin sorumlulukları

| Bileşen                                      | Sorumluluk                                                                                                                                  | Saf mı?     |
|----------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|-------------|
| Program                                      | WPD oturumunu açar, ağacı özyinelemeli yürür, başarısızları bir kez yeniden dener, bekçiyi işletir, oturumu kapatır. COM'a dokunan tek yer. | Hayır (COM) |
| IScanSink                                    | Taramanın bulgularını raporladığı sözleşme. Tarama ekrana yazmaz; bu arayüze bildirir.                                                      | —           |
| ConsoleScanSink                              | Bulguları girintili ağaç olarak basar, toplamları tutar. Girintiyi yoldan türetir.                                                          | Evet        |
| FileClassifier                               | Uzantiya, yetmezse ilk 12 byte'a bakarak dosyanın türünü belirler.                                                                          | Evet        |
| FolderPolicy                                 | Bir klasöre girilip girilmeyeceğine karar verir (OS engelli, ön bellek, çöp, çıkartma).                                                     | Evet        |
| SessionHealthMonitor                         | İçerik okumalarının hata oranını izler; oturum bozulduysa vazgeçer (devre kesici).                                                          | Evet        |
| DeviceObject ,<br>ScanError ,<br>ScanOutcome | Değişmez veri taşıyıcıları. Eksik bilgi null ile temsil edilir, uydurma varsayımlarla değil.                                                | Evet        |

## 4 Çalışma Akışı

Bir tarama beş adımdan geçer. Bekçi (watchdog) ayrı bir iş parçacığında koşar ve yalnızca **ilerlemeyi** izler — süreyi değil. Bu ayrım kritiktir: yavaş ama ilerleyen bir tarama ile donmuş bir tarama aynı görünür, ancak ilkinde müdahale edilmemeli, ikincisi ise dakikalar içinde sonlandırılmalıdır.



Şekil 2 — Tarama akışı ve bekçi. Tarama her durumda bir veridiyle biter; sessizce eksik kalması tasarım gereği imkânsızdır.

## 5 Kullanılan Tasarım Kalıpları

Kalıplar önceden seçilip probleme uydurulmadı; her biri somut bir sorun çözülürken ortaya çıktı ve sonradan adlandırıldı. En az bunun kadar önemli olan, **bilinçli olarak kullanılmayan** kalıplardır.

| Kalıp                                    | Nerede                                                  | Çözdüğü sorun                                                                                                                                                                                                         |
|------------------------------------------|---------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Circuit Breaker</b>                   | <code>SessionHealthMonitor</code>                       | Bozuk bir oturumda binlerce başarısız içerik okuması yerine, 20'lik bir pencerede %75 hata görüldüğünde vazgeçmek. Nygard'ın <i>Release It!</i> kitabındaki kalıbın el yapımı hali.                                   |
| <b>Watchdog</b>                          | <code>Program</code> (bekçi iş parçacığı)               | Donmuş ile yavaş taramayı ayırt etmek. Süreyi değil ilerlemeyi izler; 30 dakikalık genel zaman aşımının bu ayrımı yapamadığı ölçümle gösterildi.                                                                      |
| <b>Dependency Inversion / Sink</b>       | <code>IScanSink</code>                                  | Tarama bir soyutlamaya rapor verir. Konsol bugün, SQLite yarın — tarama kodu değişmez. Veritabanı ekrandan okuyamaz; bu dikiş sonraki aşamanın önkoşulu.                                                              |
| <b>Functional core, imperative shell</b> | <code>FileClassifier</code> , <code>FolderPolicy</code> | Karar veren kodu, dünyaya dokunan koddan ayırmak. Bu ayrım yapılmadan projenin en kritik mantığı telefon olmadan test edilemiyordu.                                                                                   |
| <b>Deferred Retry</b>                    | <code>RunRetryPass</code>                               | Satır içi yeniden deneme ölçüldü ve <i>kötüleştirdiği</i> görüldü (5 dk → 20+ dk): kalıcı olarak okunamayan nesnelere yapılan her deneme boşa bekleyiştir. Tek bir ertelenmiş geçiş, ana taramayı asla bloklamaz.     |
| <b>Immutable records</b>                 | <code>DeviceObject</code> vb.                           | Veri taşıyan tipler değiştirilemez; "cihaz söylemedi" <code>null</code> ile temsil edilir, <code>0</code> ya da <code>DateTime.MinValue</code> gibi ölçülmüş gibi görünen uydurma değerlerle değil.                   |
| <b>Illegal states unrepresentable</b>    | <code>FileKind</code>                                   | "Bakılmadı" ile "medya değil" tek bir değerle temsil edildiğinde 11.480 dosya sessizce kayboldu. Üç ayrı durum, üç ayrı anlam: <code>Unknown</code> , <code>Undetermined</code> , ve karar öncesi <code>null</code> . |
| <b>Idempotent cleanup</b>                | <code>CloseSession</code>                               | Üç ayrı yoldan (finally, <code>ProcessExit</code> , Ctrl+C) ulaşılabilen temizlik, <code>Interlocked</code> ile tek kazanana indirgendi.                                                                              |

### Bilinçli olarak kullanılmayanlar

Bağımlılık enjeksiyonu konteyneri, repository kalıbı, tek gerçekleştirmesi olan bir `IDeviceConnection` arayüzü (tek örneğe göre arayüz tasarlamak tahmindir), baştan sona `async/await` (WPD senkron ve iş parçacığına bağlıdır; `async` makinesi yalan olurdu). Bir kalıbı *ne zaman uygulamamak gerektiğini* bilmek, uygulamayı bilmekten daha nadir bir beceridir.

## 6 Karşılaşılan Teknik Zorluklar ve Çözümler

### 6.1 Hata alan klasörün "dosya" sanılması

Bir nesnenin özellikleri okunamadığında kod `("unreadable")`, `false` döndürüyordu. O `false` "klasör değil" anlamına geliyor, dolayısıyla nesne dosya sanılıp içine hiç girilmiyordu. Sonuç: **DCIM dahil 9 kök klasör**, hiçbir hata mesajı olmadan görünmez oldu. Kamera fotoğraflarının tamamı taramada yoktu. Çözüm: üç durumlu dönüş (`ok` bayrağı), başarısız nesnenin sınıflandırılmaması.

### 6.2 Süre üzerine dört yanlış teori

Tarama bazen 18 saniye, bazen 15 dakika sürüyordu. Sırasıyla "cihaz yorgun", "property sayısı", "round trip sayısı" ve "önbellek klasörleri" suçlandı — **dördü de yanlış** ve hepsi aynı hatadan kaynaklanıyordu: iki koşunun duvar saatini karşılaştırıp farka sebep uydurmak. Cevap, sürecin içine kronometre konunca geldi:

| Durum                               | GetValues / çağrı | Toplam   |
|-------------------------------------|-------------------|----------|
| Soğuk (bağlantı sonrası ilk tarama) | ~20 ms            | 4 – 5 dk |
| Sıcak (arka arkaya tarama)          | ~0,2 ms           | 18 sn    |

100 kat fark, ve belirleyici olan tek değişken bu. Property sayısı kontrollü A/B ile ölçüldü: 2 property 0,17 ms, 6 property 0,20 ms — **bedava**. Aynı 20 ms'lik soğuk maliyet, farklı üreticinin cihazında (Turkcell T70, ZTE yongası) bağımsız olarak doğrulandı: bu MTP'nin genel davranışdır, üreticiye özgü değil.

#### Çıkarılan kural

Farklı koşulların duvar saatini karşılaştırma. Çağrı başına ölçüm yap, yalnızca aynı önbellek durumundaki koşulları kıyasla. Bu kural koda yorum olarak gömüldü.

### 6.3 PTP modu: sessiz %23 kayıp

Aynı telefon, aynı kablo, aynı kod; yalnızca telefonun USB ayarı değiştirildi: "Dosya aktarımı" 289 dosya, "Görüntü aktarımı" **222** dosya. Kaybolan 67 dosya: cihazdaki **bütün videolar**, 42 `.webp` ve 3 `.pdf`. PTP yalnızca durağan görüntü aktarır; telefon gerisini listeden çıkarır. Tehlikeli olan şu: bu tarama "0 hata, oturum sağlıklı" dedi. Çözüm: `WPD_DEVICE_TYPE` okunur; kamera (1) ise uyarılır ve sonuç PARTIAL işaretlenir. Her iki modda da gerçek cihazda doğrulandı.

### 6.4 Bekçi: kod incelemesinin bulamadığı bug

Bekçi ilk kez zorla tetiklendiğinde kilitlenmeyi doğru tespit etti, `Cancel()` çağırdı — ve tarama **4.838 nesne daha** devam edip yine dondu. `Cancel()` "uçuştaki işlemi" iptal eder; ama tarama tek bir uzun işlem değil, 14.500 küçük işlemdir. Biri iptal olur, döngü sonrakine geçer. Beş bağımsız inceleme bu kodu okudu ve onayladı; bug yalnızca *çalıştırınca* ortaya çıktı. Çözüm: yürüyüşün kendisinin kontrol ettiği bir dur bayrağı. Sonuç: 4.838 → 36 nesne.

### 6.5 Kurtarılamayan donma

Soğuk bir tarama 22 dakika boyunca **0,00 saniye CPU** ile bekledi: iş parçacığı bir sürücü çağrısının içinde donmuştu. Ne `Cancel()` ne dur bayrağı ulaşabildi — bayrak nesneler *arasında* kontrol edilir, çağrının içinden dönmeyen bir iş parçacığı oraya hiç gelmez. Normal `kill` bile süreci sonlandıramadı. Çözüm: bekçi 20 saniyelik

bir bekleme sonrasında elindeki sayımı raporlar ve `FailFast` ile çıkar. 30 dakika beklemek bir güvenlik ağı değil, uzun adlı bir donmadır.

## 6.6 Yanlış sınıflandırmalar

Otomatik testler, canlı taramanın "sağlıklı" dediği kodda ölçülebilir hatalar buldu: `"BMW service log"` metni fotoğraf sayılıyordu (BMP kontrolü 2 byte'tı); `RIFF...WAVE` ses dosyaları medya sayılıyordu (8. byte'taki alt tip hiç okunmuyordu); `"the ftype is"` metni video sayılıyordu (kutu uzunluğu doğrulanmıyordu). Kullanıcının `Android` adlı bir albümü, OS klasörü sanılıp sessizce budanıyordu. Hepsi düzeltildi ve testle sabitlendi.

## 7 Test Stratejisi

İki tamamlayıcı katman kullanıldı. Saf mantık birim testleriyle, donanıma dokunan kod gerçek cihazlarla doğrulanır. Saf mantık `MediaTransfer.Core` adlı ayrı bir kütüphanedir; bu kütüphane bilinçli olarak hiçbir COM referansı taşımaz. İddia böylece yorumla değil derlemeyle korunur: .NET Core MSBuild'in `ResolveComReference` görevi yoktur (MSB4803), dolayısıyla bu katmana bir COM bağımlılığı sızsa testler hiç çalışmaz olurdu.

| Test dosyası                              | Kapsadığı davranışlar                                                                                                                                                                                                                                                                                                                                              |
|-------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>FileClassifierNameTests</code>      | Her uzantı kategorisi; <code>null</code> (bilinmiyor, byte'lara bak) ile <code>Unknown</code> (kesinlikle ilgisiz) ayrımı; emoji, Türkçe karakter, uzantısız ad, sondaki nokta/boşluk, <code>.trashed-</code> adları; <b>tr-TR kültüründe</b> büyük/küçük harf güvenliği — Türkçe <code>İ</code> sorunu, her <code>.JPG</code> 'yi cihaz okumasına gönderebilirdi. |
| <code>FileClassifierSignatureTests</code> | Her sihirli byte imzası (JPEG, PNG, GIF, BMP, WEBP, ftyp, TIFF/DNG, WebM, JXL, JP2, PDF); kısa tampon; yanlış pozitifler — <code>"BMW service log"</code> , <code>RIFF...WAVE</code> , uzantısız M4A.                                                                                                                                                              |
| <code>FolderPolicyTests</code>            | <code>Android/data</code> kuralının depolama köküne sabitlenmesi ( <code>/Phone/DCIM/Android</code> atlanmamalı); ters bölü ayrıçlı yollar; ön bellek, çöp ve çıkartma klasörleri; <code>.Statuses</code> gibi gerçek medya içerebilen gizli klasörlerin atlanmaması.                                                                                              |
| <code>SessionHealthMonitorTests</code>    | 14/20 tetiklemez, 15/20 tetikler; pencere sıfırlanır; bir kez tetiklenince kalır ve susar; atlanan kontroller sayılır.                                                                                                                                                                                                                                             |

### Testlerin bulduğu gerçek hatalar

BMP 2 byte, RIFF alt tipi, ftyp kutu uzunluğu, Android kuralı sabitlenmemiş. Hiçbiri canlı taramada görünmüyordu.

### Testlerin geri döndürdüğü hatalı düzeltmeler

İki kez: BMP'ye tek yönlü sınır konmuştu; sondaki nokta yanlış yerde kırılmıştı. Kırmızı testler işini yaptı.

**Birim testiyle kapsanmayan:** COM tarafı — retry geçişi, akış okuma döngüsü, oturum kapatma, bekçi. Fiziksel telefon gerektirdikleri için birim testi yazılamaz. Bunun yerine beş gerçek cihazda doğrulama ve `PROBE_FORCE_STALL` kancasıyla iptal yolunun zorla tetiklenmesi kullanıldı; bu kanca, kod incelemesinin bulamadığı bekçi hatasını ortaya çıkaran testtir.

## 8 Ölçüm Sonuçları

### 8.1 Ana cihazda (Samsung Galaxy A56) başlangıç ve son durum

| Ölçüt                      | Başlangıç                   | Son                  |
|----------------------------|-----------------------------|----------------------|
| Tarama süresi (sıcak)      | 4 dk 29 sn                  | 18 – 21 sn           |
| Cihaz hatası               | 3.136                       | 0                    |
| Görülen kök klasör         | 8 / 17                      | 16 / 16              |
| DCIM (kamera)              | görünmüyor                  | 4.005 dosya          |
| Belgeler                   | 1                           | 164                  |
| Medya olarak sayılan       | 17.416 (%46'sı küçük resim) | 13.466 (tümü gerçek) |
| Türkçe dosya adı           | sessizce bozuluyor (ğ→g)    | doğru                |
| Tarama eksikse bildirir mi | hayır                       | COMPLETE / PARTIAL   |

### 8.2 Cihazlar arası doğrulama

| Cihaz              | Özellik                                         | Dosya  | Süre         | Sonuç            |
|--------------------|-------------------------------------------------|--------|--------------|------------------|
| Samsung Galaxy A56 | Yeni Android, tek depolama                      | 13.790 | 18 sn – 4 dk | COMPLETE, 0 hata |
| Samsung J7 Prime2  | Eski Android, SD kart, 3 depolama nesnesi       | 358    | 9,6 sn       | COMPLETE, 0 hata |
| Turkcell T70       | ZTE yongası, farklı üretici                     | 91     | 16 sn        | COMPLETE, 0 hata |
| Samsung Tab S7 FE  | Tablet                                          | 62     | 1,1 sn       | COMPLETE, 0 hata |
| Kobo eReader       | E-kitap okuyucu — MTP değil, USB yığın depolama | 772    | —            | COMPLETE, 0 hata |

Cihazlar arası öğrenilenler: depolama adı üç farklı biçimde geliyor ("Dahili depolama", "Phone", "Dahili depolama birimi") — asla anahtar olarak kullanılmamalı; seri numarası formatı 8 ile 32 hane arasında değişiyor — opak string olarak tutulmalı; aynı uygulama (WhatsApp) eski ve yeni Android'de tamamen farklı yerlerde duruyor — klasör adına değil uzantıya göre sınıflandırma kararı bununla doğrulandı.

**Kobo bulgusu — imza yolunun asıl sınavı.** Telefonlarda içerik imzası neredeyse hiç çalışmaz: A56'da 13.791 dosyanın yalnızca 5'inde çalıştı. Kobo'da ise 772 dosyanın 710'unda çalıştı ve bulunan 620 medya dosyasının tamamı yalnızca imzayla yakalandı — hiçbirinin uzantısı tanınmıyordu. Yalnızca uzantıya bakan bir sınıflandırma bu cihazda sıfır dosya bulurdu. Telefonlarda "güvenlik ağı" gibi duran ve %99,96 oranında hiç çalışmayan bu yol, burada tek işleyen mekanizma oldu.

**İkinci bulgu — taşıma katmanı bağımsızlığı.** Kobo, MTP değil USB yığın depolama konuşur (cihaz kimliğinde `usbstor#disk` görülür). Kod MTP'ye özgü varsayım yapmadığı, WPD soyutlamasına yazıldığı için taşıma protokolü tamamen değiştiği hâlde tarama değişmeden, hatasız çalıştı. Bu, "MTP ayrıntılarına değil WPD'ye kod yaz" kararının sinanmış karşılığıdır.

## 9 Öğrenilen Dersler

1. **Kod incelemesi gerekli ama yeterli değil.** Beş inceleme turu, bekçinin işe yaramadığını göremedi. Çalıştırmak gördü. Güvenlik-kritik bir yolu, ona inanmadan önce tetikle.
2. **Ölçüm, kanıdan üstündür.** Süre konusunda dört yanlış teori, dördü de emin bir tonla dile getirildi; dördü de ölçümle çürüdü. Kontrolsüz karşılaştırma, hiç karşılaştırmamaktan kötüdür.
3. **Sessiz eksiklik, gürültülü hatadan kötüdür.** "0 hata, sağlıklı" diyen bir tarama %23 dosya kaybettirebiliyordu. Sistem artık tam olmadığını söylemek zorunda.
4. **Amaç netliği kapsamı değiştirir.** "Her medyayı al" ile "kaybedilse üzülmenecek olanı al" iki farklı uygulamadır. İkincisi kullanıcının gerçekten istediğidir ve 13.989 dosyalık yığını anlamlı bir kümeye indirir.
5. **Tek cihazda doğruluk, doğruluk değildir.** Dört cihaz, üç farklı depolama adı, iki seri numarası formatı, iki WhatsApp konumu.

## 10 Sonraki Adımlar

Tarama katmanı, bir sonraki aşamanın önkoşullarını karşılıyor: sonuçlar bir arayüze raporlanıyor ve her tarama tam mı kısmi mi olduğunu bildiriyor. Sıradaki aşama **SQLite tabanlı kalıcı indekstir**. Bu indeks iki başlangıç sorununu aynı anda çözer: hangi dosyanın kopyalandığını kaydeden bir manifest (sorun 1), ve klasör yapısını düzleştiren tek bir "tüm fotoğraflar" görünümü (sorun 2). Tasarım kararları alındı: cihaz kimliği seri numarasıyla anahtarlanır; dosya kimliği yol + boyut + tarih üçlüsüyle, kesinlik gerektiğinde içerik hash'iyle; kısmi bir tarama asla silinmiş dosya çıkarımına yol açmaz; tanınmayan bir cihazda hiçbir kayıt silinmez.

Ardından sırasıyla: WinUI 3 arayüzü, parçalı ve devam ettirilebilir aktarım motoru, kurulum paketi. Uygulama tasarım gereği **salt okunurdur**; telefondan silme yeteneği, ancak doğrulanmış kopya, tek tek dosya, açık kullanıcı onayı ve denetim kaydı şartlarıyla ve ayrı bir aşamada eklenebilir.

### Danışmanlık istenen açık sorular

- **Kimlik anahtarı:** Dosya kimliği için yol + boyut + tarih üçlüsü seçildi; `PERSISTENT_UNIQUE_ID` yeniden bağlantıda korunsa da oturum kimliği de korunduğundan test ayırt edici değildi. Bu tercih güvenli mi, yoksa içerik hash'i tarama anında hesaplanmalı mı?
- **Kilitlenen iş parçacığı:** Sürücü çağrısında donan bir iş parçacığı süreç içinden kurtarılamıyor; tek çıkış `FailFast`. Bir masaüstü uygulamasında bunun kabul edilebilir bir tasarım olup olmadığı — ya da taramanın ayrı bir süreçte koşturulmasının doğru olup olmadığı.
- **Performans:** Soğuk tarama ~20 ms/çağrı ile 4-5 dakika. `IPortableDevicePropertiesBulk` bunu toplu okumaya çevirebilir ama her sürücü desteklemiyor. Bu yatırım SQLite indeksinden önce mi sonra mı yapılmalı?
- **Test kapsamı:** COM tarafı birim testiyle kapsanamıyor; sahte cihaz (fake) yazmak tek gerçekleştirmeye göre arayüz tasarlamak anlamına geliyor. Bu noktada beklemek mi, yoksa hata enjeksiyonlu sahte cihaza şimdi yatırım yapmak mı doğru?

## 11 Geliştirme Süreci ve Çalışma Yöntemi

Proje, önceden yazılmış bir tasarımı uygulamak yerine **yinelemeli ve ölçüm odaklı** bir yöntemle yürütüldü: her adımda bir varsayım kuruldu, gerçek cihazda sınandı, sonuç ya varsayımı doğruladı ya da düzeltti. Aşağıda bu yöntemin somut unsurları özetlenmiştir.

### 11.1 Karar alma

Mimari ve ürün kararları geliştirici tarafından, seçeneklerin artı ve eksileri anlaşıldıktan sonra verildi. Başlıca kararlar: C#/.NET ve WPD COM API'sinin seçimi (Rust/Tauri ve Python alternatiflerine karşı); Android'in iPhone'dan önce ele alınması; SQLite'in LiteDB yerine tercih edilmesi (sektörde yaygınlık ve öğrenme değeri gerekçesiyle); WinUI 3'ün WPF yerine seçilmesi; uygulamanın **salt okunur** olması ve silme yeteneğinin kesin kurallara bağlanması; "her medyayı al" yerine "kaybedilse üzülmünecek olanı al" kapsamının benimsenmesi; çıkartmaların dışlanıp ekran görüntüleri ile WhatsApp resimlerinin tutulması.

### 11.2 Gerçek donanımla doğrulama

Hiçbir düzeltme yalnızca "mantıklı görüldüğü" için kabul edilmedi. Her değişiklik bağlı bir telefonda çalıştırıldı ve sonuç sayılarla karşılaştırıldı. Dört farklı cihaz (iki üretici, telefon ve tablet, eski ve yeni Android, SD kartlı ve kartsız) kullanıldı. Kablo sökme-takma, farklı USB portuna geçme, USB modunu MTP ile PTP arasında değiştirme gibi fiziksel deneyler bizzat yürütüldü ve bunların her biri kodun tek başına ortaya çıkaramayacağı bir bulgu üretti: PTP modunun sessiz veri kaybı, cihaz kimliğinin port değişiminde korunması, soğuk ön belleğin 100 kat maliyeti.

### 11.3 Varsayımların sorgulanması

Süreçte birden çok kez, sunulan teşhisler kanıt istenerek reddedildi. "Cihaz yorgun" açıklaması ölçüm olmadığı gerekçesiyle kabul edilmedi ve daha sonra yanlış olduğu kanıtlandı. Eklenen tanılama kodunun kalıcı hale gelmesine karşı çıkıldı ve her geçici parça açık bir kaldırma işaretiyle etiketlendi. Bekçi mekanizmasının "çalıştığı" iddiası, gerçekten tetiklenmesi istenerek sınandı; bu test, beş inceleme turunun göremediği bir hatayı ortaya çıkardı.

### 11.4 Araçlar ve iş akışı

Geliştirme, bir yapay zekâ kod asistanıyla eşli programlama biçiminde yürütüldü: kodun yazımı ve tekrarlayan işler asistana bırakılırken, her kararın gerekçesi açıklanması istendi ve anlaşılmadan hiçbir parça kabul edilmedi. Asistanın hatalı sonuçlara vardığı durumlar (süre üzerine dört yanlış teori, DCIM teşhisi, bekçinin işlevselliği) geliştirici tarafından yakalanıp ölçümle düzeltildi. Kod incelemesi için birbirinden bağımsız açılardan bakan paralel inceleme turları düzenlendi; bulgular tek tek doğrulanıp önceliklendirildi. Sürüm kontrolü için Git kullanıldı; her commit, değişikliğin *neden* yapıldığını ve hangi ölçümle doğrulandığını açıklar. Tüm kod, yorumlar ve commit mesajları İngilizce yazıldı.

### 11.5 Öğrenme hedefi

Projenin amacı çalışan bir program üretmenin ötesinde, her parçanın *neden* öyle olduğunu anlamaktı. Bu nedenle kolay yol olan "çalışıyorsa dokunma" yaklaşımı yerine, her hata kök nedenine kadar izlendi ve her düzeltme ya bir birim testiyle ya da gerçek cihaz ölçümüyle sabitlendi. Raporun 9. bölümündeki dersler, bu sürecin doğrudan ürünüdür.

## 12 Yönetici Özeti

### Ne yapıldı

Android telefonlardaki fotoğraf ve videoları bilgisayara güvenli biçimde aktaracak bir Windows uygulamasının **cihaz tarama katmanı** tasarlandı, gerçekleştirildi ve beş gerçek cihazda doğrulandı. Katman, telefona MTP protokolüyle bağlanır, tüm dosya ağacını yürür, her dosyayı uzantı ve içerik imzasıyla sınıflandırır, güvenilir değilse donanım karşısında hatasız çalışmaya devam eder ve **sonucunun tam mı eksik mi olduğunu açıkça bildirir**.

### Neden önemli

Mevcut ticari araçların temel sorunu, aktarımın yarıda kesildiğinde hangi dosyanın kopyalandığını bilmemeleridir. Bu sorunun kökü, uygulamanın cihazda ne olduğunu güvenilir biçimde bilmemesidir. Çalışma boyunca ölçüldü ki bir tarama "0 hata, oturum sağlıklı" raporlarken bile cihazdaki tüm videoları kaçırabiliyor (PTP modu) ya da kamera klasörünün tamamını görmezden gelebiliyor (hata alan klasörün dosya sanılması). Bu nedenle katmanın birincil tasarım hedefi hız değil **dürüstlük** oldu: eksik bir tarama tam görünemez.

### Mimari

Altı bileşen, tek ilkeyle ayrıştırıldı: **donanıma dokunan kod ile karar veren kod ayrı yaşar**. Sınıflandırma ( `FileClassifier` ), klasör politikası ( `FolderPolicy` ) ve sağlık izleme ( `SessionHealthMonitor` ) saf sınıflardır; COM, cihaz ya da konsol bilmezler. Tarama, sonuçlarını bir arayüze ( `IScanSink` ) raporlar; bugün konsol, bir sonraki aşamada SQLite bu arayüzün gerçekleştirmeleridir. Kullanılan kalıplar: devre kesici, bekçi, bağımlılığın tersine çevrilmesi, işlevsel çekirdek / buyurgan kabuk, ertelenmiş yeniden deneme, değişmez veri tipleri. Bilinçli olarak kullanılmayanlar: DI konteyneri, repository kalıbı, tek gerçekleştirmeli soyut arayüzler.

### Test

291 birim testi, 205 milisaniyede, telefon olmadan çalışır ve saf mantığın tamamını kapsar. Bu testler, canlı taramanın sağlıklı dediği kodda yedi ölçülebilir hata buldu ve iki hatalı düzeltmeyi geri döndürdü. Donanıma dokunan kod beş gerçek cihazla ve zorla tetiklenen hata senaryolarıyla doğrulandı. Saf mantık, COM referansı taşımayan ayrı bir kütüphanede durur; bu katmana bir COM bağımlılığı sızsa testler derlenemez olurdu, yani "bu sınıflar saftır" iddiasını derleyici kanıtlar.

### Sonuçlar

| Ölçüt                          | Başlangıç  | Son                        |
|--------------------------------|------------|----------------------------|
| Tarama süresi (sıcak önbellek) | 4 dk 29 sn | 18 sn                      |
| Cihaz hatası                   | 3.136      | 0                          |
| Görülen kök klasör             | 8 / 17     | 16 / 16                    |
| Kamera klasörü (DCIM)          | görünmüyor | 4.005 dosya                |
| Eksik tarama tespiti           | yok        | COMPLETE / PARTIAL verdiği |
| Doğrulanmış cihaz              | 1          | 5                          |

### Başlıca bulgular

- Tarama süresi, istek sayısı ile değil **sürücü önbelleğinin durumuyla** belirlenir: bağlantı sonrası ilk tarama 100 kat yavaştır. Bu, kalıcı bir indeksin (SQLite) performans için de zorunlu olduğunu gösterir.

- Kilitlenen bir sürücü çağrısı süreç içinden kurtarılamaz; `Cancel()` ve iptal bayrağı ulaşamaz. Bekçi 65 saniyede tespit edip elindeki sayımı raporlayarak çıkar.
- Kod incelemesi, çalıştırmanın yerini tutmaz: beş inceleme turunun onayladığı bekçi mekanizması, ilk gerçek tetiklemede işe yaramadığını gösterdi.

### Sıradaki adım

SQLite tabanlı kalıcı indeks. Önkoşulları karşılandı: tarama bir arayüze raporlar ve tamlığını bildirir. Şema kararları alındı: cihaz seri numarasıyla, dosya yol + boyut + tarih üçlüsüyle anahtarlanır; kısmi tarama asla silinmiş dosya çıkarımına yol açmaz. Danışmanlık istenen dört açık soru 10. bölümde listelenmiştir.

## 13 Üç Soruya Doğrudan Cevap

Bu sayfa, istenen üç soruyu tek sayfada, raporun ilgili bölümlerine atıfla cevaplar.

### Soru 1 — Adım adım ne yapıldı?

- Cihaza bağlanma:** WPD COM API'siyle telefon bulundu, salt okunur oturum açıldı, seri numarası ve USB modu okundu.
- Ağaç yürüyüşü:** Tüm klasörler özyinelemeli gezildi; her dosya önce uzantıyla, tanınmazsa ilk 12 byte'ın imzasıyla sınıflandırıldı.
- Güvenilmez donanım:** Hata alan nesne atlanıp devam edildi; başarısızlar ana tarama bitince bir kez yeniden denendi (satır içi deneme ölçülüp kötüleştirdiği görüldü, kaldırıldı).
- Oturum sağlığı:** İçerik okuma hata oranı izlendi, oturum bozulduysa vazgeçilip raporlandı. Kapatılmayan oturum sonraki taramayı bozuyordu; kapatma ve COM temizliği eklendi.
- Kod incelemesi:** İki tur, 20+ kusur. En ağırı: hata alan klasörün dosya sanılıp DCIM dahil 9 kök klasörün sessizce kaybolması; Türkçe adları bozan kodlama hatası.
- Ölçüm:** Süre için dört yanlış teori; süreç içi kronometre gerçek sebebi buldu (sürücü önbelleği, 100×). Property okumanın bedava olduğu kontrollü A/B ile kanıtlandı.
- Ürün gerçekleri:** Küçük resim ve önbellek klasörleri (medyanın %46'sı) ile çıkartmalar dışlandı; PTP modunun sessiz veri kaybı (289 → 222 dosya) tespit edilip uyarı eklendi.
- Bekçi:** Donmayı ilerlemeye bakarak tespit eder; zorla tetiklenince `Cancel()` 'ın döngüyü durdurmadığı görüldü, dur bayrağı eklendi; kurtarılamayan donmada sayım raporlanıp çıkılır.
- Ayrıştırma ve test:** Tek dosya 6 parçaya bölündü; saf mantık 291 birim testiyle, donanım tarafı 5 gerçek cihazla doğrulandı. Tarama artık COMPLETE / PARTIAL verdiği veriyor.

Ayrıntı: Bölüm 6, 11.

### Soru 2 — Tasarım kalıbı kullanıldı mı?

**Evet, sekiz tane** — ancak hiçbirini önceden seçilip probleme uydurulmadı; her biri bir sorunu çözerken ortaya çıkıp sonradan adlandırıldı. **Circuit Breaker** (bozuk oturumda vazgeçme), **Watchdog** (ilerlemeye dayalı donma tespiti), **Dependency Inversion** ( `IScanSink` : tarama bir arayüze raporlar, konsol ve SQLite onun gerçekleştirmeleridir), **Functional core / imperative shell** (karar veren kod ile donanım dokunan kod ayrı), **Deferred Retry**, **Immutable records**, **Illegal states unrepresentable** ( `FileKind` 'da "bakılmadı" ile "medya değil" ayrı değerler), **Idempotent cleanup**.

**Bilinçli olarak kullanılmayanlar:** DI konteyneri, repository kalıbı, tek gerçekleştirmesi olan soyut arayüz, baştan sona `async/await` . Gerekçeleri Bölüm 5'te.

### Soru 3 — Hangi testler var?

| Katman                                                 | Ne var                                                                                                                                                                                                                                                                  | Ne buldu                                         |
|--------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------|
| <b>Birim testleri</b><br>198 test · 55 ms · telefonsuz | Uzantı ve imza sınıflandırması (her format, yanlış pozitifler, emoji, Türkçe, uzantısız), klasör politikası (Android/data sabitlemesi, önbellek, çıkartma), devre kesici eşiği ve durum geçişleri. Test projesi kaynakları bağlayarak derlenir; COM bağımlılığı yoktur. | 7 gerçek hata; 2 hatalı düzeltmeyi geri döndürdü |
| <b>Gerçek cihaz doğrulaması</b><br>5 cihaz             | Samsung A56, J7 Prime2 (SD kartlı), Turkcell T70 (ZTE), Tab S7 FE, Kobo eReader (USB yığın depolama). Her düzeltme sonrası tam tarama, sayılar karşılaştırıldı. Kablo sökme-takma, port değiştirme, MTP/PTP geçişi.                                                     | PTP kaybı, port kararlılığı, soğuk önbellek      |
| <b>Zorlamalı hata senaryosu</b><br>PROBE_FORCE_STALL   | Bekçi kasıtlı olarak tetiklenip iptal → geri sarma → PARTIAL yolu uçtan uca doğrulandı.                                                                                                                                                                                 | Cancel()'ın döngüyü durdurmadığı                 |

Kapsanmayan: COM tarafı birim testiyle test edilemez (telefon gerekir); bu boşluk gerçek cihaz doğrulamasıyla kapatıldı. Ayrıntı: Bölüm 7.

## 14 Kod Envanteri

Raporun yazıldığı andaki tam durum: 11 dosya, 2.945 satır. **PRE-SQLITE** etiketi, yalnızca kalıcı depolama olmadığı için var olan ve SQLite aşamasında silinecek parçaları işaretler.

### Proje yapısı

```
Device Probe/
├── Device Probe/                ← ana proje (Exe, COM referanslı)
│   ├── Program.cs              1164  yürütme: WPD oturumu, tarama, retry, bekçi, özet
│   ├── FileClassifier.cs       194   saf: uzantı + sihirli byte sınıflandırma
│   ├── ConsoleScanSink.cs     102   konsol çıktısı + toplamalar
│   ├── FolderPolicy.cs        96    saf: klasör atlama kuralları
│   ├── SessionHealthMonitor.cs 71   saf: devre kesici
│   ├── IScanSink.cs           68    sözleşme + ScanOutcome
│   └── FileKind.cs            36     enum
└── Device Probe.Tests/         ← xunit, kaynakları LINKLER, COM'suz
    ├── FileClassifierNameTests 29 test
    ├── FileClassifierSignatureTests 22 test
    ├── FolderPolicyTests       17 test
    └── SessionHealthMonitorTests 16 test (Theory'ler açılınca toplam 198)
```

**Derleme notu:** ana proje `dotnet build` ile derlenmez (MSB4803, COM referansı) → VS MSBuild şart. Test projesi `dotnet test` ile çalışır.

### Program.cs — akış sırasıyla

| #  | Ne                                                                                                                                                              | Satır   | Durum / Not                                                                                                  |
|----|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|--------------------------------------------------------------------------------------------------------------|
| 1  | <code>Console.OutputEncoding = UTF8</code>                                                                                                                      | 10      | Türkçe ad koruması, ölçüldü                                                                                  |
| 2  | <code>GetDevices</code> — ilk cihazı al                                                                                                                         | 30-39   | <b>Bilinen sınırlama:</b> çoklu cihazda sessizce ilkini seçer; düzeltmesi elle yazılmış interop gerektiriyor |
| 3  | Cihaz adı, <code>deviceId</code> yazdır                                                                                                                         | 43-61   | <code>deviceId</code> iki USB portunda aynı ölçüldü                                                          |
| 4  | Seri no / model / üretici key'leri                                                                                                                              | 70-76   | SQLite <code>device</code> tablosunun girdisi                                                                |
| 5  | <code>device.Open()</code> + client info + <code>GENERIC_READ</code>                                                                                            | 79-106  | Salt okunur oturum                                                                                           |
| 6  | PROPERTYKEY tanımları (name, contentType, originalFileName, size, persistentId, dateModified, deviceType)                                                       | 112-195 | <code>PARENT_ID</code> bilerek istenmiyor (yürüyüş zaten biliyor)                                            |
| 7  | <code>wantedProperties</code> koleksiyonu (6 key)                                                                                                               | ~185    | Ölçüldü: 6 key ≈ 2 key maliyeti                                                                              |
| 8  | İmza sayaçları ( <code>caughtBySignatureOnly</code> , <code>signatureChecksActuallyRun</code> , <code>signatureCheckErrors</code> )                             | 204-214 | <b>PRE-SQLITE</b> (üçüncüsü yarı kalır)                                                                      |
| 9  | <code>sink</code> , <code>health</code> örnekleri                                                                                                               | 216-217 |                                                                                                              |
| 10 | <code>failedObjects</code> — retry çalışma kümesi (ObjectId, Path, Stage)                                                                                       | 230     | Kalır                                                                                                        |
| 11 | <code>classifiedObjectIds</code> / <code>walkedContainerIds</code>                                                                                              | 237-242 | İlki <b>PRE-SQLITE</b> , ikincisi kalır (döngü kırma)                                                        |
| 12 | <code>sessionClosed</code> , bekçi durumu ( <code>lastProgressTicks</code> , <code>stallDetected</code> , <code>scanFaulted</code> , <code>scanAborted</code> ) | 245-256 | Kalır                                                                                                        |
| 13 | <code>filePropertyMisses</code>                                                                                                                                 | 258-265 | <b>PRE-SQLITE</b> ; artık sadece dosyaları sayıyor (düzeltildi, cihazda doğrulanmadı)                        |
| 14 | Seri/model/üretici oku, yazdır                                                                                                                                  | ~268    |                                                                                                              |

|    |                                                                                                               |              |                                                                                                  |
|----|---------------------------------------------------------------------------------------------------------------|--------------|--------------------------------------------------------------------------------------------------|
| 15 | <code>IsCameraMode()</code> → <code>sink.OnScanStarted( ... )</code>                                          | ~272         | PTP tespiti; <b>zincir cihazda hiç tetiklenmedi</b>                                              |
| 16 | <code>ProcessExit</code> + <code>CancelKeyPress</code> → <code>CloseSession</code>                            | ~280         |                                                                                                  |
| 17 | <code>scanTask = Task.Run(PrintTree + RunRetryPass)</code>                                                    | ~285         |                                                                                                  |
| 18 | <b>Bekçi thread'i</b> : 45 sn ilerlemesizlik → bayrak + <code>Cancel()</code> → 20 sn → <code>FailFast</code> | ~290-400     | Zorlamalı testle doğrulandı; gerçek kilitlenmede <b>FailFast</b> öncesi özet <b>doğrulanmadı</b> |
| 19 | <code>PROBE_FORCE_STALL</code> env kancası                                                                    | bekçi içinde | Bilinçli test aracı, kalıcı                                                                      |
| 20 | 30 dk genel timeout + <code>Cancel()</code> + <code>FailFast</code>                                           | ~360-400     | Bekçi varken nadiren tetiklenir, yedek                                                           |
| 21 | <code>try/catch(AggregateException)/finally{CloseSession; OnScanFinished}</code>                              | ~400-470     | Her çıkış yolunda verdi bildirilir                                                               |
| 22 | <b>Özet bloğu</b> : Done / imza istatistikleri / health / <b>[NOTE]</b> / atlanan klasörler / hata gruplama   | ~590-690     | Tamamı <b>PRE-SQLITE</b> → <b>ConsoleScanSink</b> 'e taşınacak                                   |

## Program.cs — fonksiyonlar

| Fonksiyon                                             | Satır       | Ne yapar                                                                                                                 | Not                                                                              |
|-------------------------------------------------------|-------------|--------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------|
| <code>RunRetryPass</code>                             | 469         | Başarısızları bir kez yeniden dener; klasöre yeniden yürür; kilitlenmişse atlar                                          | <code>walkedContainerIds</code> 'e bakar (çift <code>OnFolder</code> düzeltildi) |
| <code>CloseSession</code>                             | 687         | COM release zinciri + <code>device.Close()</code> , <code>Interlocked</code> ile tek kazanan                             | 3 yoldan ulaşılır                                                                |
| <code>PrintTree</code>                                | 721         | Özyinelemeli yürüyüş; iptal bayrağı, ilerleme sinyali, <code>FolderPolicy</code> , sink çağrıları                        | <code>depth</code> parametresi kaldırıldı                                        |
| <code>ReadDeviceString</code>                         | 842         | Cihaz düzeyi string property                                                                                             |                                                                                  |
| <code>IsCameraMode</code>                             | 863         | <code>WPD_DEVICE_TYPE == 1</code>                                                                                        | Kamera=1, medya oynatıcı=2, telefon=3 ölçüldü                                    |
| <code>ClassifyAndReportFile</code>                    | 895         | Dedup guard → <code>FileClassifier.ClassifyByName</code> → imza → <code>sink.OnFile</code>                               |                                                                                  |
| <code>CheckSignatureWithHealthMonitoring</code>       | 917         | Devre kesici sarmalayıcısı; kapalıysa <code>Undetermined</code>                                                          |                                                                                  |
| <code>DetectKindBySignature</code>                    | 951         | <code>GetStream</code> + 12 byte döngülü okuma + <code>ClassifyBySignature</code> ; <code>bytesReadPtr</code> sıfırlanır | COM olmayan hata artık taramayı öldürmez                                         |
| <code>TryReadString</code> / <code>TryReadSize</code> | 1067 / 1078 | Opsiyonel property; <code>expected</code> param ile konteynerleri saymaz                                                 |                                                                                  |
| <code>GetObjectInfo</code>                            | 1089        | <code>GetValues</code> → <code>DeviceObject?</code> (null = başarısız)                                                   | Üç durumlu; "(unreadable)" sentineli gitti                                       |

## Saf sınıflar

**FileClassifier** (194 satır) — 4 uzantı kümesi (medya 27, ses 10, bilinen-medya-değil 22, belge 7); `ClassifyByName` → `FileKind?` (null = byte'lara bak); `.trashed-` / `.pending-` eleme; sondaki nokta/boşluk kırpma;

**ClassifyBySignature** — 12 imza (JPEG, PNG, GIF, BMP+boyut, RIFF/WEBP, ftyp+uzunluk+marka, TIFF LE/BE, WebM, JXL×2, JP2, PDF); **Matches** yardımcı.

**FolderPolicy** (96 satır) — **Android/data|obb** (depolama köküne sabitli, / ve \); önbellek sözlüğü ( **.thumbnails** , **.Links** , **.wamocache** , **.Trash** ); **\*Sticker\*** alt-string; null güvenli.

**SessionHealthMonitor** (71 satır) — 20'lik pencere, %75 eşik; **RecordResult** → tetiklenince oran döner, sonra susar; **RecordSkippedCheck** ; **CheckingDisabled** , **SkippedBecauseDisabled** .

## Sözleşme ve veri

**IScanSink** — **OnScanStarted**(id, name, camera) , **OnFile**(obj, path, kind, recovered) , **OnFolder** , **OnFolderSkipped**(path, reason) , **OnError**(ScanError) , **OnScanFinished**(ScanOutcome) .

**ScanOutcome** — **Completed** , **Stalled** , **Faulted** , **CameraMode** , **UndeterminedFiles** , **SubtreeLosses** .

**ConsoleScanSink** — girintiyi yoldan türetir; **[MEDIA]/[DOC]/[UNCHECKED]/[DIR]/[SKIP]** ; **COMPLETE/PARTIAL** verdiği; **Errors / SkippedFolders** listeleri (**PRE-SQLITE**).

**DeviceObject** — **ObjectId** , **Name** , **DisplayName** , **IsContainer** , **Size?** , **PersistentId?** , **ModifiedRaw?** .

**ScanError** — **ObjectId** , **ParentPath** , **Stage** , **HResult** , **Message** . **ScanStage** — **Enumerate** , **EnumerateNext** , **Properties** .

**FileKind** — **Unknown** (bakıldı, ilgisiz) , **MediaFile** , **Document** , **Undetermined** (bakılamadı).

## Testler (198)

| Dosya                                    | Kapsam                                                                                                                                                                        |
|------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>FileClassifierNameTests</b> (29)      | Her uzantı kategorisi; null/Unknown ayrımı; emoji, Türkçe, uzantısız, . / .. , sondaki nokta/boşluk, 300 karakter; <b>tr-TR kültüründe</b> büyük/küçük harf; <b>.trashed-</b> |
| <b>FileClassifierSignatureTests</b> (22) | Her imza; kısa tampon; sıfır tampon; yanlış pozitifler ( <b>BMW service</b> , <b>RIFF WAVE</b> , <b>the ftype is</b> , M4A markası); eksik format imzaları                    |
| <b>FolderPolicyTests</b> (17)            | Android kuralı sabitleme ( <b>/Phone/DCIM/Android</b> atlanmaz); \ yollar; önbellek; çıkartma; <b>.Statuses</b> atlanmaz; null; 500 seviye derin yol                          |
| <b>SessionHealthMonitorTests</b> (16)    | 14/20 hayır, 15/20 evet; pencere sıfırlanır; tetiklenince kalır ve susar; atlanan sayılır                                                                                     |

**Kapsanmayan:** COM tarafının tamamı (**PrintTree**, **retry**, **bekçi**, **CloseSession**, **GetObjectInfo**, **DetectKindBySignature**). Telefon gerektirir.

## Bilinen sınırlamalar (kodda yorum olarak yazılı)

- Çoklu cihazda ilkinin sessizce seçer (elle interop gerekir)
- Kilitlenen sürücü çağrısı süreç içinden kurtarılamaz → **FailFast** tek çıkış
- PERSISTENT\_UNIQUE\_ID** kalıcılığı belirsiz — tasarım ona güvenmiyor
- Tarih string olarak geliyor ( **2026/02/18:15:44:29.000** ), parse edilmiyor
- Sadece 4 cihazda denendi (3 Samsung + 1 ZTE)

## Bilinçli ertelenenler

**IDeviceObjectSource** + sahte cihaz · COM'un **WpdSession** 'a taşınması · **IPortableDevicePropertiesBulk** · **resume/kurtarma** · **IDeviceConnection** · özet bloklarının sink'e taşınması (SQLite ile birlikte)

## Doğrulanmamış

- filePropertyMisses** 3 → 0 tahmini (cihaz yok)
- PTP zinciri ( **OnScanStarted** uyarısı → **PARTIAL/CameraMode** )
- FailFast** öncesi özet (gerçek kilitlenme gerekir)

## 15 Çalışma Notları — Kavramlar

Bu bölüm, proje boyunca karşılaşılan teknik kavramları, her birini projenin kendi koduna ve yaşanan olaylara bağlayarak açıklar. Amaç, "ne kullandık" listesinden öte, her kavramın *neden* gerektiğini ve yanlış anlaşıldığında neye mal olduğunu anlatmaktır.

### 15.1 COM — Component Object Model

Microsoft'un 1993'te tasarladığı bir sistem: bir programın, başka bir programın (veya Windows'un kendisinin) sunduğu nesneleri, hangi dille yazılmış olursa olsun, yeniden derlemeden kullanabilmesi için ortak bir kural seti.

**Benzetme:** standart bir elektrik prizi. Prizin şeklini ve voltajı herkes biliyor; hangi markanın fişi olursa olsun takılır, çalışır. COM da nesneler için böyle bir "priz standardı" — Windows bir yeteneği COM nesnesi olarak sunar, C# yazan da, C++ yazan da aynı prize takılır.

**Bizim projede:** Windows'un "USB'deki telefonla MTP üzerinden konuş" yeteneği bir COM API olarak sunuluyor — WPD (Windows Portable Devices). Kodumuzdaki şu satırların hepsi COM çağrısıdır:

```
new PortableDeviceManagerClass() // C# 'new' gibi görünür ama Windows'a "cihaz yöneticini ver" diyor
device.Open(deviceId, clientInfo) // Windows'un nesnesindeki metodu çağırıyoruz
content.EnumObjects(...) // sürücü bunu USB üzerinden telefona iletiyor
```

**Normal C#'tan dört farkı vardır ve dördü de bizi uğraştırdı:**

- Çöp toplayıcı yok, referans sayacı var.** C#'ta nesneyi bırakırsın, çöp toplayıcı temizler. COM nesnesi "kaç kişi beni tutuyor" diye sayar; sayı sıfıra inince kendini yok eder. Sayıyı *bizim* düşürmemiz gerekir: `Marshal.ReleaseComObject(childIds)`. Unutulunca her nesnede bir COM nesnesi açılıp hiç bırakılmadı, binlerce sızıntı oluştu, oturum kilitli kaldı ve sonraki tarama "device unreachable" verdi. `CloseSession` 'daki release zinciri ve sırasının önemi bundandır (ayrıntı 15.4).
- C# doğrudan konuşamıyor, çevirmen var.** COM'un dili C/C++ tarzıdır; .NET araya otomatik bir sarmalayıcı üretir ( `PortableDeviceApiLib` ). Çevirmen mükemmel değildir: `GetDevices` 'ın dizi alması gerekirken `ref string` alması, `Next` 'in tek seferde bir nesne dönmesi bundandır (ayrıntı 15.3).
- Hatalar sayı olarak gelir.** COM metodu başarısız olunca `0x80070141` gibi bir kod döndürür (HRESULT); .NET bunu `COMException` 'a çevirir (ayrıntı 15.5).
- Thread kuralları vardır.** COM nesneleri "apartment" denen thread gruplarına aittir; yanlış gruptan çağrılınca ya çalışmaz ya da sessizce yavaşlar (ayrıntı 15.6).

**"COM tarafı test edilemiyor" neden:** bu nesneler bizim değil, Windows'un ve USB sürücüsünün. `PrintTree` çağrıldığında gerçekten bir telefonla konuşur. Sahte telefon yaratmanın yolu araya kendi arayüzümüzü koyup testte taklit etmektir; bunu tek gerçek implementasyona göre arayüz tasarlamak tahmin olacağı için erteledik. Bugün "COM tarafı" dediğimiz şey `Program.cs` 'te Windows nesnelerine dokunan ~51 satır ve etrafındaki yürüyüş mantığıdır; gerisi COM bilmez ve 55 milisaniyede test edilir.

### 15.2 WPD, MTP ve PTP

**MTP (Media Transfer Protocol)**, telefonun USB üzerinden konuştuğu protokoldür. Bir dosya sistemi *değildir*: telefon, her birinin opak bir kimliği olan "nesne"lerden oluşan bir ağaç sunar. Üç tür soru sorabilirsin — "X nesnesinin çocuklarını listele", "X'in özelliklerini ver", "X'in byte'larını ver" — ve her soru bir USB gidiş-dönüşüdür. Kök nesnenin adı "DEVICE", altındaki ilk seviye depolama birimleridir (A56'da 1, J7'de 3).

**WPD (Windows Portable Devices)**, Windows'un MTP'yi programlara sunduğu COM katmanıdır. Biz MTP'yi hiç görmeyiz; WPD'nin nesnelerini çağırırız, sürücü onları MTP komutlarına çevirir.

**PTP (Picture Transfer Protocol)**, MTP'nin atası ve fotoğraf makineleri için tasarlanmış dar bir alt kümesidir: yalnızca durağan görüntü, klasör yapısı yok. Telefon "Görüntü aktarımı" modundayken PTP konuşur. Ölçtük: aynı J7, aynı kablo, MTP 289 dosya, PTP 222 — kaybolan 67 dosya cihazdaki bütün videolar, 42 `.webp` ve 3 PDF. Tarama ise "0 hata, sağlıklı" dedi; çünkü telefon o dosyaları listeden çıkarmıştı, biz sormamıştık bile. Bu yüzden `WPD_DEVICE_TYPE` okunur ve kamera (1) ise sonuç PARTIAL işaretlenir.

### 15.3 Interop ve çevirmen (RCW, marshalling)

.NET ile COM arasındaki köprüye **interop** denir. Derleme zamanında `tlbimp` aracı, COM tip kütüphanesini okuyup her COM arayüzü için bir C# sınıfı üretir; projedeki `PortableDeviceApiLib` ve `PortableDeviceTypesLib` bunlardır. Çalışma zamanında her COM nesnesinin yerine bir **RCW** (Runtime Callable Wrapper) durur: C# tarafında normal bir nesne gibi görünür, arkada gerçek COM nesnesine yönlendirir.

Parametrelerin iki dünya arasında dönüştürülmesine **marshalling** denir. Sorun, çevirmenin her yapıyı doğru çevirememesidir. Yaşadıklarımız:

- `GetDevices(ref string, ref uint)` — native imza bir *dizi* ister; çevirmen tek `ref string` üretti. Cihaz sayısını öğrenmek için dokümandaki "NULL geçir" deseni bu imzayla ifade edilemez; denedik, cihaz tespiti tamamen bozuldu. Çoklu cihaz desteği bu yüzden "elle yazılmış interop gerekir" diye bekliyor.
- `IEnumPortableDeviceObjectIDs::Next` — aynı sebeple tek seferde bir nesne alıyoruz; batch mümkün ama elle imza ister.
- `ref _tagpropertykey` — struct'ları referansla geçirmek COM'un beklediği şekildir; doğru çalışıyor.
- `IStream` — `tlbimp`'in ürettiği versiyon değil, .NET'in `ComTypes.IStream` 'i kullanılmalı; cast ederek geçiyoruz, tek RCW, ölçüldü.

### 15.4 Referans sayma ve kaynak temizliği

COM nesnesinin iç sayacı vardır; `Marshal.ReleaseComObject` bunu bir düşürür, sıfıra inince nesne yok olur. Bu üç kural doğurur:

1. **Her aldığı bırak, finally içinde.** `GetObjectInfo` her nesnede bir `IPortableDeviceValues` alır; `PrintTree` her klasörde bir enumerator alır. 19.000 nesnede bunlar bırakılmayınca sızıntı oluştu. `finally` bloğu, hata olsa da bırakılmasını garanti eder.
2. **Sıra önemlidir.** Oturumdan türeyen nesneler ( `content` , `properties` , `resources` ) önce, sonra `device.Close()` , sonra `device` , en son `deviceManager` . Tersini yapınca "COM object that has been separated from its underlying RCW cannot be used" hatası alındı — gerçekten alındı.
3. **Bir kez bırak.** `CloseSession` üç yoldan çağrılabilir (normal bitiş, Ctrl+C, süreç çıkışı). İki aynı anda girerse bir nesne iki kez bırakılır ya da biri kullanırken diğeri yok eder. `Interlocked.Exchange` ile yalnızca ilk giren kazanır (15.6).

Ölçüm: süreç canlı izlendi — handle sayısı 7 dakika boyunca 284-293 arasında sabit, bellek ~10 MB büyüdü. Sızıntı yok. Bu ölçüm, kalan hataların bizim değil sürücünün olduğunu kanıtladı.

### 15.5 HRESULT ve COMException

COM metotları başarı/hata bilgisini 32 bitlik bir sayıyla döndürür. En yüksek bit 1 ise hata ( `0x8...` ). `0x8007xxxx` biçimi Windows'un genel hata kodlarını sarar; `0x802Axxxx` WPD'ye özgüdür. .NET, hata dönen her çağrıyı `COMException` olarak fırlatır; `ex.HResult` ile sayıya ulaşılır.

| Kod        | Anlamı                                              | Bizde ne zaman görüldü                                                       |
|------------|-----------------------------------------------------|------------------------------------------------------------------------------|
| 0x80070141 | Cihaz erişilemez                                    | Bozuk oturumda her imza kontrolü; DCIM listelenemediğinde                    |
| 0x80070079 | Semafor zaman aşımı — cihaz cevap vermedi           | Kilitlenme öncesi                                                            |
| 0x8007001F | Cihaz çalışmıyor (hung)                             | Aynı aile                                                                    |
| 0x802A0006 | E_WPD_DEVICE_IS_HUNG — literal olarak "cihaz dondu" | Aynı aile; bu kod, sorunun istek yükü olduğunu gösterdi                      |
| 0x800703E3 | I/O uygulama isteğiyle iptal edildi                 | Cancel() çalıştığında — yani sağlıklı cihazda iptalin işe yaradığının kanıtı |

Kural: `catch (COMException)` daralt, ama *her şeyi* yakalayan bir `catch` de bırakma — narrow catch, cast hatalarını (interop sorunlarını) yakalamayınca bir dosya bütün taramayı öldürdü. Şimdi COM hataları ayrı, diğerleri ayrı ele alınır.

## 15.6 Thread'ler, apartment, Interlocked ve Volatile

Bir **thread** (iş parçacığı), programın aynı anda yürüyen bir kolu. Bizde üç tane var: ana thread (bekler), tarama thread'i ( `Task.Run` içinde yürür), bekçi thread'i (ilerlemeyi izler). Aynı değişkene birden fazla thread dokunduğunda üç sorun çıkar:

- **Görünürlük:** bir thread'in yazdığını diğeri hemen görmeyebilir (işlemci önbelleği). `Volatile.Write/Read` bunu garanti eder; `scanAborted` bayrağı bu yüzden `Volatile` ile okunur.
- **Yarış (check-then-act):** `if (!closed) { closed = true; ...temizle... }` iki thread'de aynı anda çalışırsa ikisi de `false` görüp ikisi de temizler. `Interlocked.Exchange(ref x, 1) == 1` bunu tek atomik adımda yapar: yalnızca ilk çağırın 0 görür.
- **COM apartment'ı:** COM nesneleri ya tek thread'e bağlı (STA) ya da serbesttir (MTA). Konsol uygulamasının ana thread'i ve thread havuzu MTA; WPD'nin CLSID'leri `ThreadingModel=Both` — ölçüldü, kayıt defterinden. Yani nesneler thread'ler arasında güvenle paylaşılır. Belirleyici olan thread'in apartment'ı değil, sunucunun kayıtlı modelidir.

Bir ders daha: başka bir thread'den COM nesnesini *burakmak* güvenli değildir — nesne bir çağrının ortasında iken yok edilirse access violation olur ve .NET bunu yakalayamaz. Timeout yolundaki ilk tasarım tam bunu yapıyordu; `Cancel()` tabanlı tasarıma geçildi.

## 15.7 Devre kesici (Circuit Breaker)

Elektrikteki sigortadan adını alır: bir şey sürekli başarısız oluyorsa, denemeye devam etmek yerine kes ve durumu bildir. Yazılımda Michael Nygard'ın *Release It!* kitabıyla yaygınlaşmış, Netflix Hystrix ve Polly gibi kütüphanelerle standartlaşmıştır. Bizde `SessionHealthMonitor`, elle yazılmış hali.

**Neden gerekti:** WPD oturumunun içerik okuma kanalı, özellik okuma kanalı sağlamken sessizce bozulabiliyor. Ölçüldü: 1607 imza kontrolünün 1606'sı "device unreachable" verdi ve tarama yine de "başarılı" bitti — 174 gerçek dosya kaçtı.

**Nasıl çalışır:** 20'lik pencerede hata oranı  $\geq 75\%$  ise oturum bozuk sayılır ve imza kontrolü *o tarama için* kapatılır. Kayan pencere kullanılır çünkü toplam oran, önceki başarılarla sulanıp eşige çok geç ulaşır. Bir kez tetiklenince

açılmaz: tarama ortasında yeniden bağlanma denendi ve sonuçları bozdu (19.000 yerine 17.500 dosya), çünkü özyineleme yığınındaki enumerator'lar eski oturuma aitti.

**Kritik ayrıntı:** devre kesici kapalıyken atlanan dosyalar "medya değil" olarak *değil*, `Undetermined` ("bakılamadı") olarak raporlanır. Bu ayrım yokken 11.480 dosya sessizce kayboldu (simülasyon).

## 15.8 Bekçi (Watchdog)

Bir süreç canlı mı, donmuş mu — bunu izleyen ayrı bir gözcü. Bizdeki kritik tasarım kararı: bekçi **süreyi değil ilerlemeyi** izler. Sağlıklı bir soğuk tarama 5 dakika sürebilir; donmuş bir tarama da 5 dakika sürer. İkisini ayıran, ilerleme olup olmadığıdır. Tarama her nesnede bir zaman damgası yazar; bekçi 2 saniyede bir bakar, 45 saniyedir damga güncellenmediyse cihaz donmuş demektir.

**Ölçülen gerçek:** donmuş bir tarama 22 dakika boyunca 0,00 saniye CPU kullandı — thread sürücü çağrısının içinde kilitliydi. 30 dakikalık genel zaman aşımı bu durumu ayırt edemez; bekçi 45 saniyede eder.

**Kod incelemesinin bulamadığı hata:** bekçi ilk yazıldığında `device.Cancel()` çağırıyordu ve beş inceleme "doğru" dedi. Zorla tetiklenince tarama 4.838 nesne daha devam etti. Sebep: `Cancel()` "uçuştaki işlemi" iptal eder; tarama 14.500 küçük işlemdir, biri iptal olur döngü sonrakine geçer. Çözüm: yürüyüşün her nesnede kontrol ettiği bir dur bayrağı. Sonuç 36 nesne.

**Kurtarılamayan durum:** thread bir çağrının içinde donmuşsa bayrağı hiç göremez. 20 saniyelik bekleme sonrasında bekçi elindeki sayımı raporlar ve `Environment.FailFast` ile çıkar — çünkü 30 dakika beklemek güvenlik ağı değil, uzun adlı bir donmadır. `PROBE_FORCE_STALL` ortam değişkeni bu yolu bilerek tetiklemek için bırakılmış kalıcı bir test kancasıdır.

## 15.9 Dosya imzası (magic bytes)

Her dosya biçiminin kodlayıcısı, dosyanın en başına sabit bir byte dizisi yazar: JPEG `FF D8 FF`, PNG `89 50 4E 47`, PDF `%PDF-`. Uzantı bir isimdir, değiştirilebilir; imza içeriğin kendisidir. Bu yüzden uzantısı tanınmayan dosyalarda ilk 12 byte okunur.

**Pahalı olduğu için nadir yapılır:** bir byte okuması cihaza gerçek bir içerik isteğidir (A56'da ~13 ms, T70'te 278 ms). O yüzden önce uzantı bakılır; yalnızca gerçekten bilinmeyen uzantılarda imzaya gidilir. Bilinen-ama-ilgisiz uzantılar ( `.txt`, `.db`, ses dosyaları) listelenir ki onlara hiç gidilmesin.

**Testlerin bulunduğu hatalar** — imzanın "yeterince" değil "tam" kontrol edilmesi gerekir: BMP iki byte'la kontrol edilince `"BMW service log"` metni fotoğraf sayıldı (artık beyan edilen boyut da doğrulanır); RIFF konteynerinin 8. byte'taki alt tipi okunmayınca WAVE ses dosyaları medya sayıldı (artık yalnızca `WEBP` kabul edilir); `ftyp` kutusunun uzunluğu doğrulanmayınca `"the ftype is"` metni video sayıldı. TIFF/DNG, WebM, JPEG-XL, JP2 uzantı listesindeydi ama imzası yoktu; bozuk uzantılı bir DNG görünmezdi.

## 15.10 Birim testi ve saf fonksiyon

**Birim testi**, bir fonksiyonu bilinen girdiyle çağırıp cevabın beklenen olup olmadığına bakan küçük bir kod parçasıdır. Milisaniyede çalışır, dış dünyaya (cihaz, dosya, ağ) dokunmaz.

**Saf (pure) fonksiyon**, aynı girdiye her zaman aynı çıktıyı veren ve yan etkisi olmayan fonksiyondur. Yalnızca saf fonksiyonlar bu şekilde test edilebilir; bir fonksiyon içinde `properties.GetValues(...)` varsa telefon olmadan çalışmaz. Projenin ilk halinde sınıflandırma mantığı böyle bir fonksiyonun içine gömülüydü — `"webp"` medya sayılıyor mu" sorusu ancak telefonu takıp 14.000 dosya tarayarak cevaplanıyordu.

**Ayrıştırma bunu değiştirdi:** `FileClassifier`, `FolderPolicy`, `SessionHealthMonitor` saf sınıflar oldu. 198 test 55 milisaniyede koşar. Asıl kazanç şudur: **telefonda üretemediğin durumları test edebilirsiniz.** Emoji'li dosya

adını telefon kabul etmiyor, yani o durum gerçek cihazla test edilemezdi; saf fonksiyonla tek satır. Kullanıcının `DCIM/Android` adlı albümünün budanıp budanmadığını görmek için telefona sahte klasör kurmak gerekirdi; testle anında.

**Test projesinin yapısı:** ana Exe'ye referans vermez, kaynak dosyaları *bağlar* (link). Çıktısında hiçbir COM dosyası yoktur — yani "bu sınıflar saftır" iddiasını derleyici doğrular. Bir de: `dotnet build` ana projeyi derleyemez (COM referansı için VS MSBuild gerekir), ama test projesi `dotnet test` ile çalışır.

**Testler beni de düzeltti:** BMP'ye tek yönlü boyut sınırı koydum, test kırmızı kaldı — ASCII metin 1,7 GB gibi bir değer veriyordu, üst sınır da gerekiyordu. Sondaki noktayı uzantıdan kırpıttım, test kırmızı kaldı — `Path.GetExtension("photo.jpg.")` zaten boş döndüğü için ismi kırmak gerekiyordu. Kırmızı test, yanlış düzeltmeyi üretime sokmadan yakalar.

## 15.11 Soğuk ve sıcak önbellek

Windows'un MTP sürücüsü, cihazdan okuduğu nesne bilgisini bir süre hafızasında tutar. Bağlantıdan sonraki **ilk tarama soğuktur**: her özellik gerçekten telefondan gelir, ~20 ms/çağrı, 14.500 nesne → 4-5 dakika. **Sonraki tarama sıcaktır**: sürücü hafızadan cevaplar, ~0,2 ms/çağrı, 18 saniye. Yüz kat.

**Bunun öğrettiği metodolojik ders projenin en pahalı dersiydi:** tarama süresi hakkında dört farklı teori kuruldu, dördü de emin bir tonla dile getirildi, dördü de yanlıştı. Hepsi aynı hatadan: iki koşulun duvar saatini karşılaştırıp farka sebep uydurmak — oysa koşullar önbellek durumu bakımından farklıydı. Cevap, sürecin *içine* kronometre konunca geldi: çağrı başına süre, önbellekten etkilenmeyen tek ölçüdür. Property sayısının bedava olduğu da ancak böyle kanıtlandı (2 property 0,17 ms, 6 property 0,20 ms — aynı sıcak durumda, arka arkaya).

**Ürün için anlamı:** kullanıcının yaşadığı tarama her zaman soğuk olandır. Çözüm daha az property istemek değil, aynı şeyi her seferinde sıfırdan sormamaktır — yani kalıcı bir indeks (SQLite).

## 15.12 Arayüz ve bağımlılığın tersine çevrilmesi

**Arayüz (interface)**, "şu metotları sağlayan herhangi bir şey" demektir; kim sağladığı önemli değildir. `IScanSink` altı metot tanımlar: tarama başladı, dosya bulundu, klasör bulundu, klasör atlandı, hata oluştu, tarama bitti.

**Bağımlılığın tersine çevrilmesi (Dependency Inversion):** tarama kodu `ConsoleScanSink` 'e değil `IScanSink` 'e bağımlıdır. Bugün konsol, yarın SQLite, sonra UI — tarama kodu değişmez. Bu dikiş olmadan bir sonraki aşamaya geçilemezdi, çünkü veritabanı ekrandan okuyamaz.

**Tasarım kararı:** sink, `Unknown` dosyaları da alır. Bir sink'in bir şeyi göstermemeyi seçmesi bir görüntüleme tercihidir; taramanın onu düşürmesi veri kaybıdır — ve bu proje tam olarak bu şekilde 9 kök klasör kaybetti. Bir de `OnScanFinished(ScanOutcome)` : tarama tam mı kısmi mi, bunu bilmeyen bir veritabanı yazıcısı, eksik taramayı tam sanıp "şu dosyalar silinmiş" çıkarımı yapar.

## 15.13 Değişmez veri tipleri (record)

C#'ta `record`, alanları oluşturulduktan sonra değiştirilemeyen veri taşıyıcı tanımlar. `DeviceObject`, `ScanError`, `ScanOutcome` böyledir. Değişmezlik iki şey sağlar: bir nesneyi paylaşmak güvenlidir (kimse habersiz değiştiremez), ve "bu değer nereden geldi" sorusunun tek cevabı vardır.

**Nullable'ın anlamı:** `Size?`, `ModifiedRaw?`, `PersistentId?` boş olabilir. "Cihaz söylemedi" bilgisi `null` ile temsil edilir; `0` ya da `DateTime.MinValue` gibi ölçülmüş gibi görünen uydurma değerlerle değil. Uydurma değer, eksik veriyi sessizce manifest'e sızdırır.

## 15.14 Git ve commit mesajı

Her commit'in mesajı, deęişiklięin *ne* olduęunu deęil *neden* yapıldıęını ve hangi ölçümle doęrulandıęını anlatır. "Fix bug" yerine: hangi girdi yanlış sonuç veriyordu, mekanizma neydi, düzeltme sonrası ölçüm ne oldu. Bunun sebebi pratiktir: altı ay sonra kodu okuyan kiři (muhtemelen sen), *neyin* deęiřtięini diff'ten görür; *nedenini* yalnızca mesajdan öğrenir. Yanlış teoriler de kaydedilir — "bu koşuda řunu sandık, ölçüm çürüttü" — çünkü aynı tuzaęa ikinci kez düşmemenin yolu budur.

Tüm kod, yorumlar ve commit mesajları İngilizce; portföy amacıyla.